HANDS-ON SECURITY PROJECT

Sentinel SOC Lab

Detecting, Triaging, and Hunting Attacks with Microsoft Sentinel and Defender XDR

A step-by-step walkthrough, explained in plain language

What this is

I built a small, working security monitoring setup on Microsoft Azure. Then I attacked my own test computer, twice, and watched the alarms go off. After that I investigated each alarm the way a security analyst would. This document walks through every step in simple words, with screenshots taken from the real lab.

BUILT WITH

Microsoft Sentinel | Microsoft Defender XDR | Defender for Endpoint | Azure Monitor Agent | KQL | MITRE ATT&CK

BUILT

September 2026, on a Microsoft Azure trial subscription

THE SHORT VERSION

What I built, and why

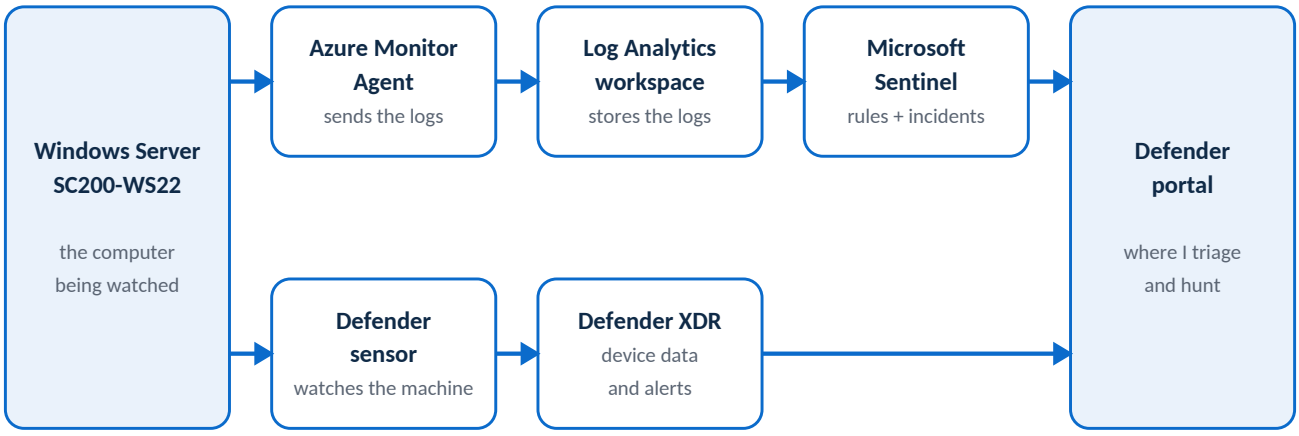
I am preparing for the Microsoft SC-200 exam (Security Operations Analyst). Reading about security tools only gets you so far, so I wanted a project that creates **real alerts** I could investigate, not screenshots from a tutorial.

So I set up a Windows Server in the cloud and connected it to **Microsoft Sentinel**, a tool that collects security logs and raises alarms. I wrote two alarm rules. Then I played the attacker against my own machine, two different ways. Both attacks set off the alarms. I opened each alarm, checked the evidence, worked out exactly what had happened, and closed it with a proper note.

What I ended up with:

- A Windows Server that is watched around the clock, and locked down so only my own connection can reach it.
- Its security logs flowing into Microsoft Sentinel.
- Microsoft Defender for Endpoint protecting the server, set to **ask me before** it fixes anything.
- Two custom detection rules that I wrote myself.
- Two real incidents, investigated and closed.
- A search query that proves **which account** was added to the Administrators group, and **who** added it.

How the pieces fit together



Two paths from the same computer end up in the same place.

If the diagram feels like a lot, here is the same thing as a picture you can hold in your head. Think of an office building with a security team:

In the lab	Think of it as
The Windows Server	The office building being watched
Logs	Security camera footage

In the lab	Think of it as
Microsoft Sentinel	The control room where all the footage arrives
A detection rule	An alarm trigger, like "ring the bell if someone fails the door code five times"
An incident	The alarm ticket that an analyst picks up and investigates
KQL (the query language)	The search bar for the footage

STEP 1

The test computer, and a locked door

Everything starts with a computer to watch. I created a **Windows Server 2022** virtual machine in Azure and named it **SC200-WS22**. It is small on purpose: 2 processors and 4 GB of memory is plenty for a lab.

Then I locked the door. **Remote Desktop (RDP)** is how you sign in to a server from far away, and bots on the internet start guessing passwords against open RDP doors within hours. So I created a network rule that lets in **only one address: mine**.

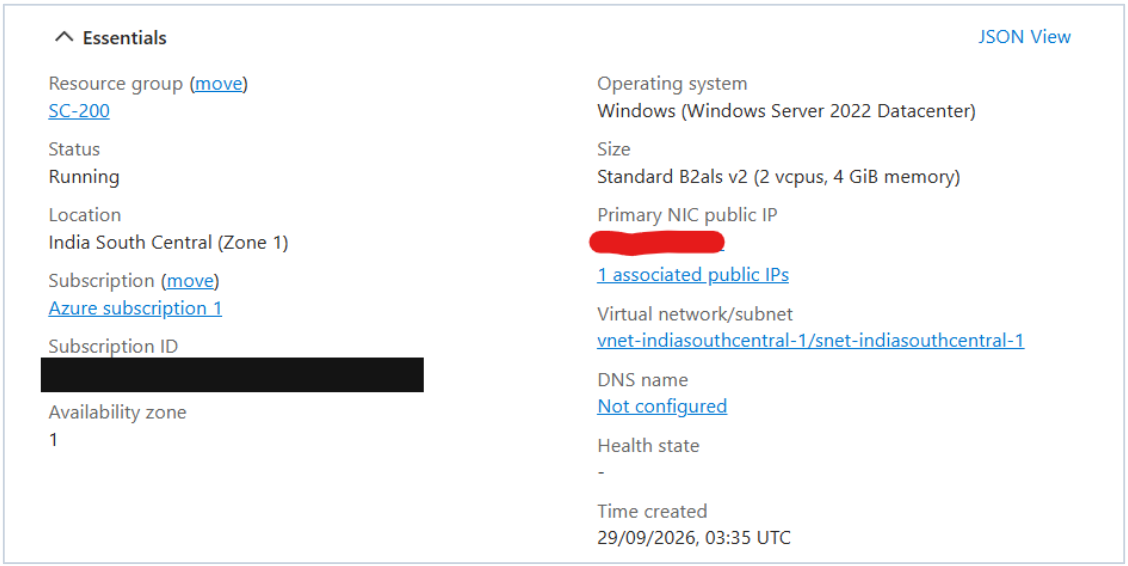


Figure 1. *The virtual machine SC200-WS22 running in Azure (Windows Server 2022, India South Central). Subscription details are hidden.*

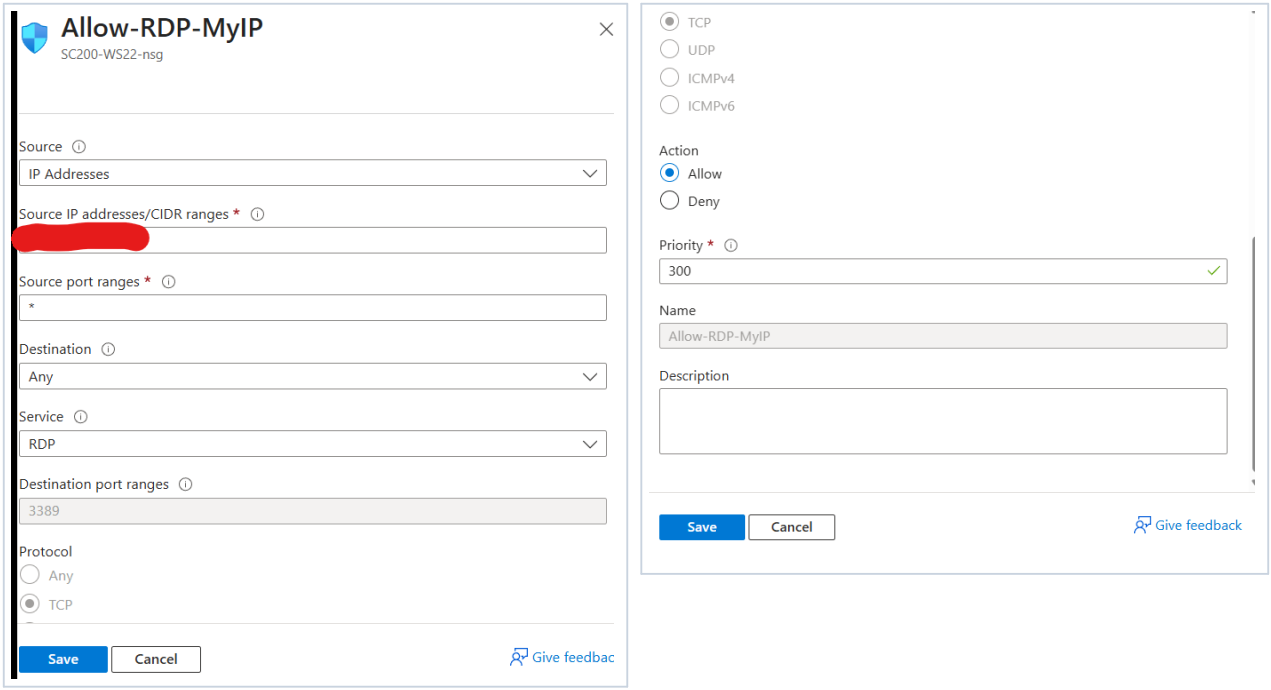


Figure 2. *The network rule called Allow-RDP-MyIP. It allows RDP (port 3389) from one source address only. My address is hidden.*

> **Why this matters**
>
> If the door is open to everyone, the logs fill up with real attacks from strangers, and my own practice attacks get lost in the noise. Locking it down kept the data clean and the server safe.

STEP 2

Sending the computer's logs to Sentinel

A server on its own tells Sentinel nothing. It has to send its logs. Windows writes a diary of everything that happens: sign-ins, failed sign-ins, new accounts, and more. I set up two small pieces to ship that diary to Sentinel:

- **Azure Monitor Agent:** a small helper program on the server that sends the logs.
- **Data Collection Rule:** a shopping list that says which logs to send and where to send them. I chose the **Common** set (the everyday security events) instead of **All events**, so I would not pay to store noise.

The logs go to a storage area called a **Log Analytics workspace** (mine is named law-soc-lab). Sentinel reads from there.

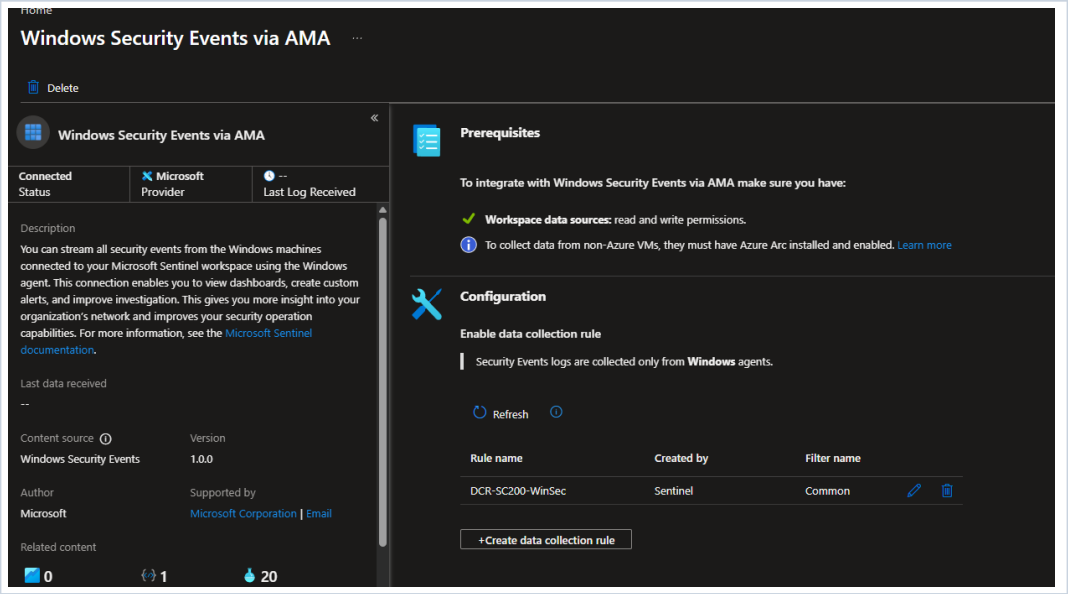


Figure 3. The Windows Security Events connector in Sentinel shows the status Connected, with my data collection rule attached.

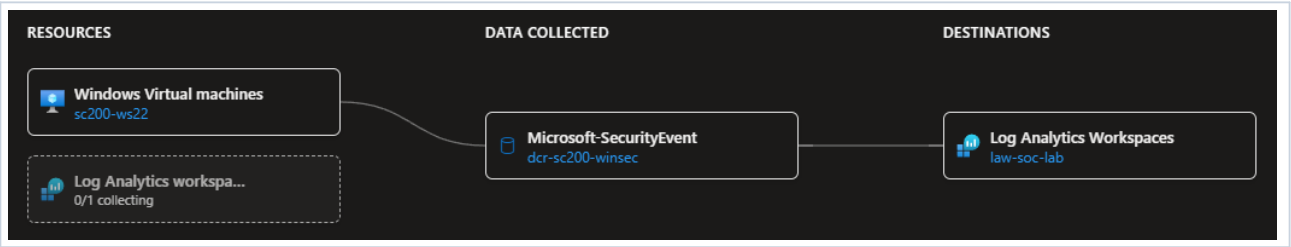


Figure 4. The pipeline in one picture: the server, the data collection rule, and the workspace where the logs land.

How I checked it worked. I did not want to assume. I ran two small searches in KQL, the language Sentinel uses to search logs. The first asks for any ten security events. The second asks when the agent last said hello.

```
SecurityEvent | take 10

Heartbeat | summarize LastSeen = max(TimeGenerated) by Computer
```

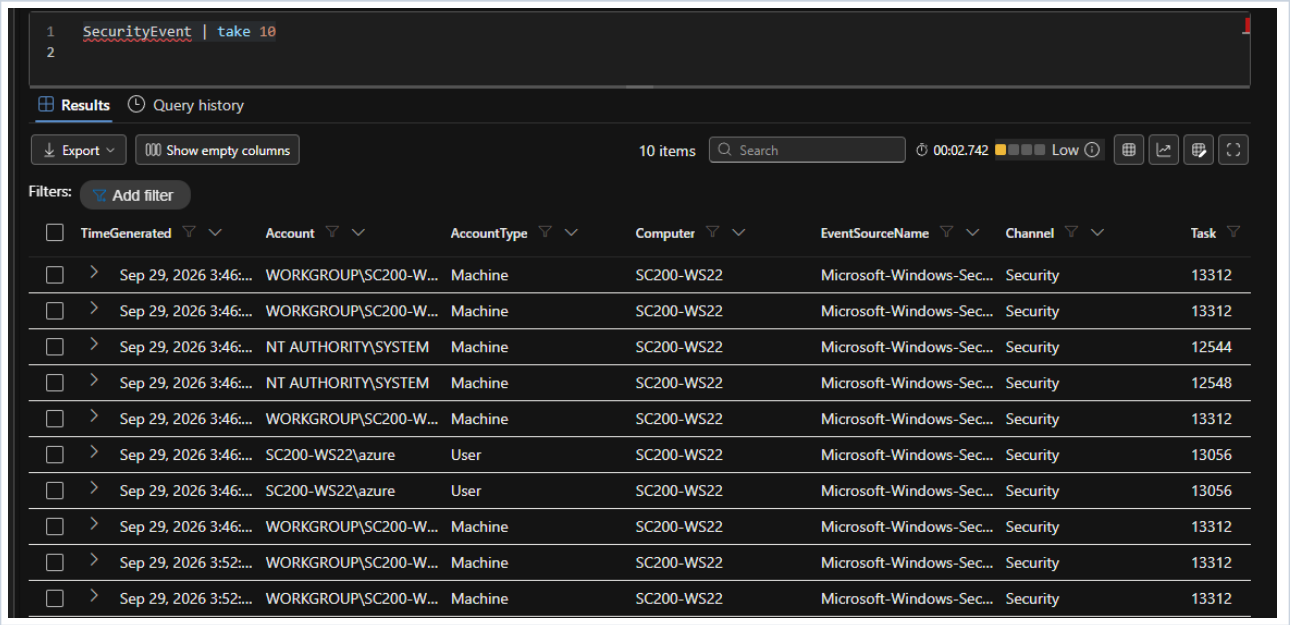


Figure 5. *Real security events arriving from SC200-WS22. This is the moment the pipeline became real.*

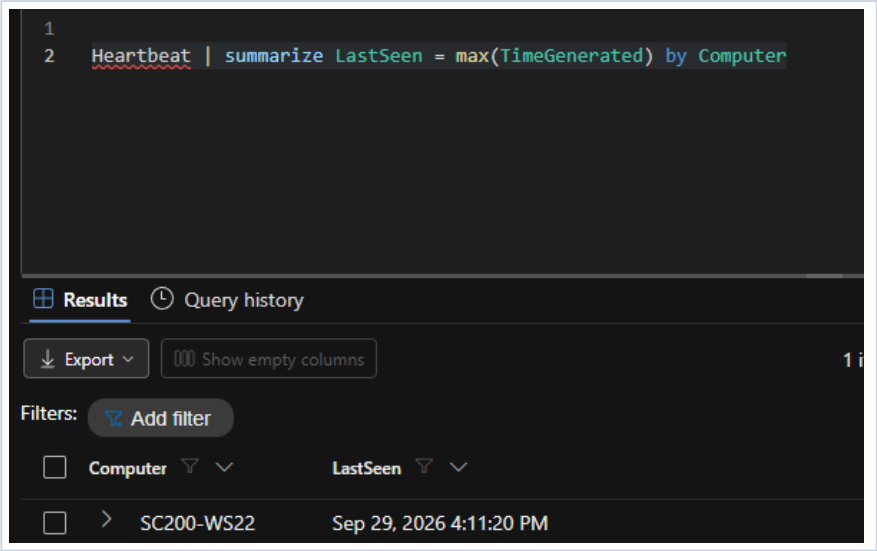


Figure 6. *The heartbeat check: the agent on SC200-WS22 is reporting in.*

STEP 3

Adding Defender for Endpoint

Logs tell you what happened. **Microsoft Defender for Endpoint** also watches the computer itself: programs that start, files that change, and signs of attack. I turned on **Defender for Servers (Plan 2)** in Defender for Cloud, and I left every other paid plan switched off to keep the cost down.

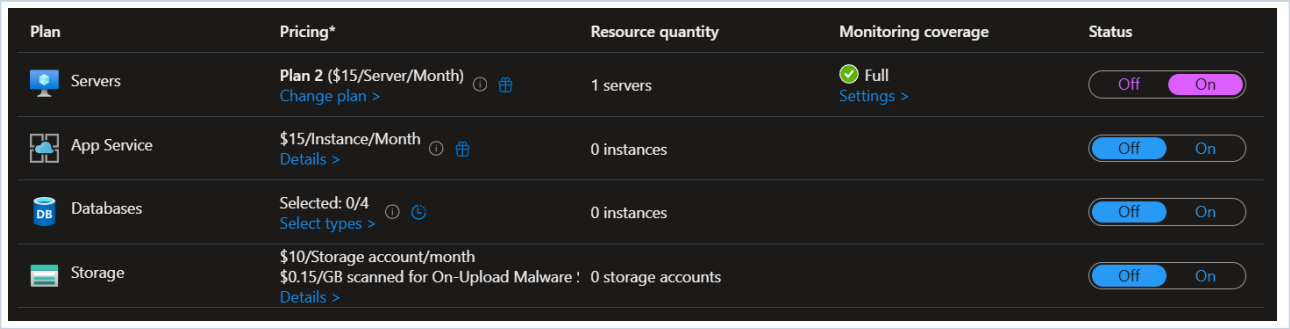


Figure 7. Defender plans: only Servers is turned on. Everything else stays off.

Once the connection was working, the server showed up in Defender's device list as a Windows Server. The sensor reports **Running**, and the value **OnboardingState = 1** means the server is properly connected.

```
PS C:\Users\azure> Get-Service Sense
>> Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows Advanced Threat Protection\Status" | Select OnboardingState

Status   Name               DisplayName
------   ----               -----------
Running  Sense              Windows Defender Advanced Threat Pr...

OnboardingState : 1
```

Figure 8. The Defender sensor is running on the server, and OnboardingState is 1.

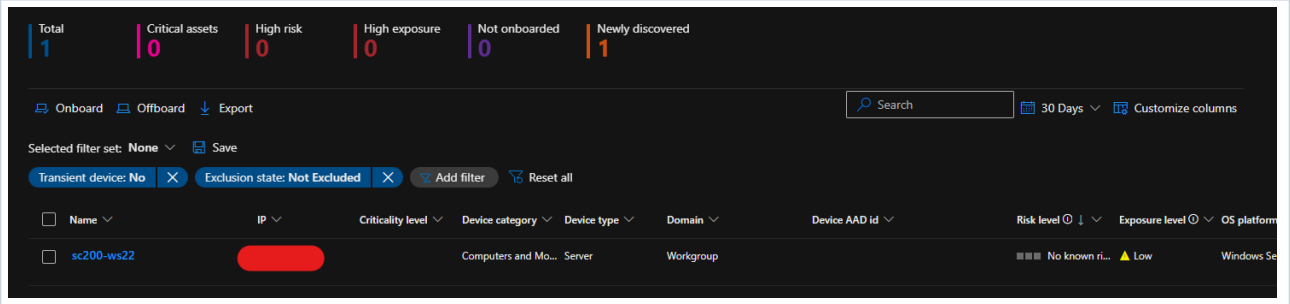


Figure 9. SC200-WS22 in the Defender device inventory: one Windows Server, no known risk, exposure level low.

Next I put the server into a **device group** called **SC200-Lab-Servers**. I set the group to **Semi - Approval required for all folders**. In plain words: Defender is allowed to investigate by itself, but it has to **ask me before it fixes or deletes anything**.

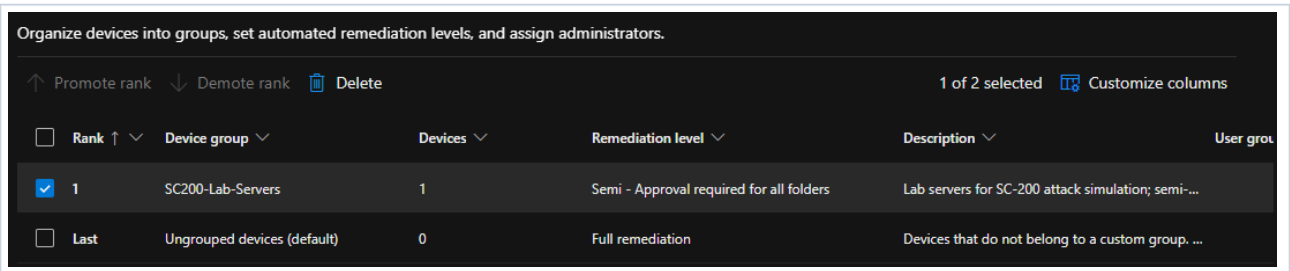


Figure 10. The device group SC200-Lab-Servers: one device, with the remediation level set to ask for approval.

> **Why "ask me first"?**
>
> In a real company, an automatic clean-up could delete a file that someone needs. Keeping a human in the loop is the safer starting point while you learn what normal looks like.

STEP 4

Writing two detection rules

A **detection rule** is a saved search that runs on a timer. If the search finds a match, it raises an alert, and Sentinel turns that alert into an **incident** for an analyst to look at. I wrote two:

Rule	What it looks for	Severity	Runs
SC200 - Brute force failed logons	The same account failing to sign in 5 or more times within 10 minutes (Windows event 4625).	Medium	Every 5 minutes
SC200 - New local admin created	A new local account being created (event 4720), or someone being added to the Administrators group (event 4732).	High	Every 5 minutes

Both rules are tagged with a **MITRE ATT&CK** tactic. MITRE ATT&CK is a public catalogue of the tricks attackers use. Rule 1 matches *Credential Access* (guessing or stealing passwords, technique T1110). Rule 2 matches *Persistence* (staying inside a system, technique T1136.001).

Both rules also tell Sentinel which part of an alert is a **computer** (Host) and which is a **person or account** (Account). This is called **entity mapping**. It is what lets the incident screen draw a picture of who and what was involved.

The search behind rule 1:

```
SecurityEvent
| where EventID == 4625
| summarize Failures = count() by TargetUserName, Computer
| where Failures >= 5
```

The search behind rule 2:

```
SecurityEvent
| where EventID == 4720 or (EventID == 4732 and TargetUserName == "Administrators")
| project TimeGenerated, Computer, EventID, Activity, TargetUserName, SubjectUserName
```

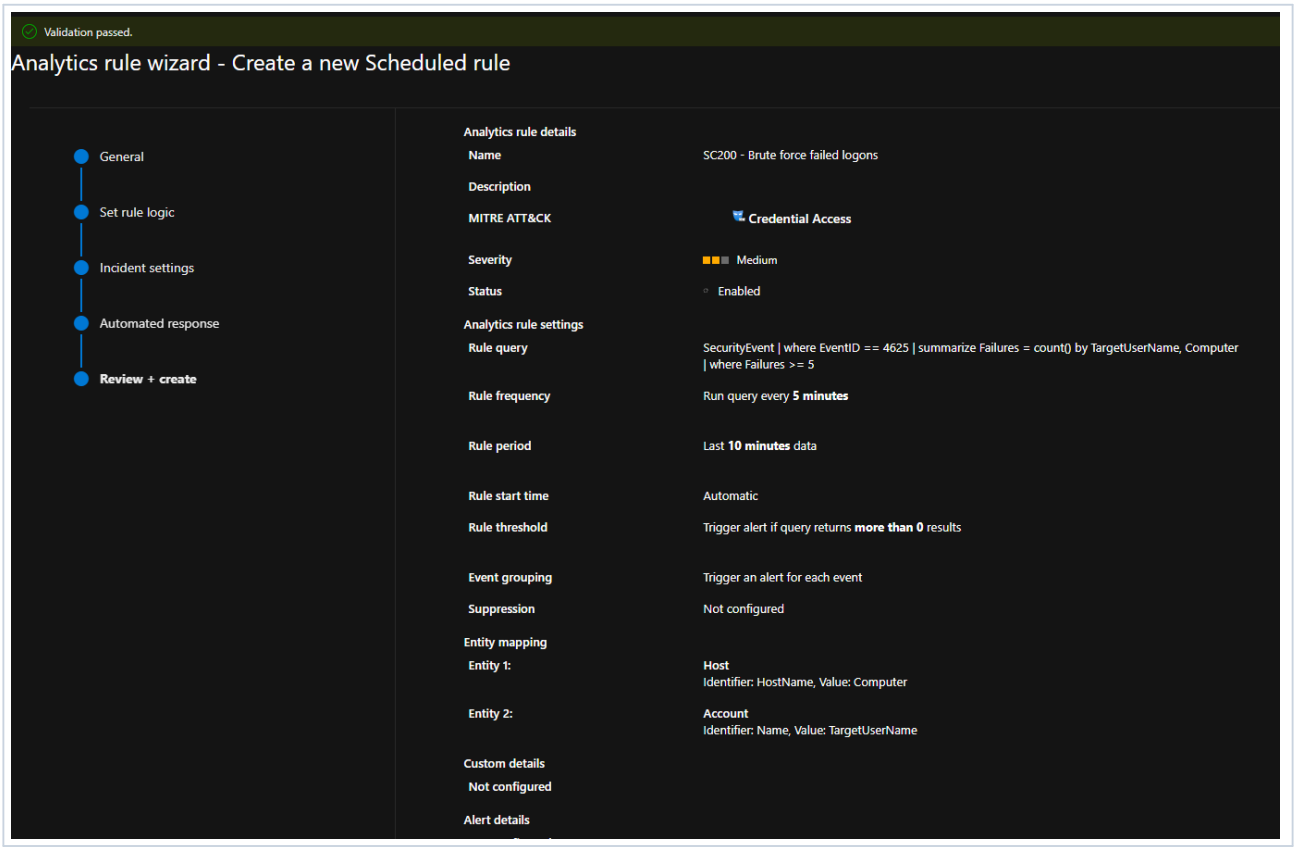


Figure 11. *Rule 1, review page: it runs every 5 minutes, looks back over 10 minutes, and maps both the Host and the Account.*



Figure 12. *Rule 2, review page: the query, the schedule, and the Host and Account mapping.*

ATTACK 1

Guessing passwords

What I did. From inside the server, I tried to sign in eight times with a wrong password, using an account that does not exist (**labfake**). That is the same pattern as someone guessing passwords.

```
1..8 | ForEach-Object { net use \\localhost\IPC$ /user:labfake WrongPass$_ }
```

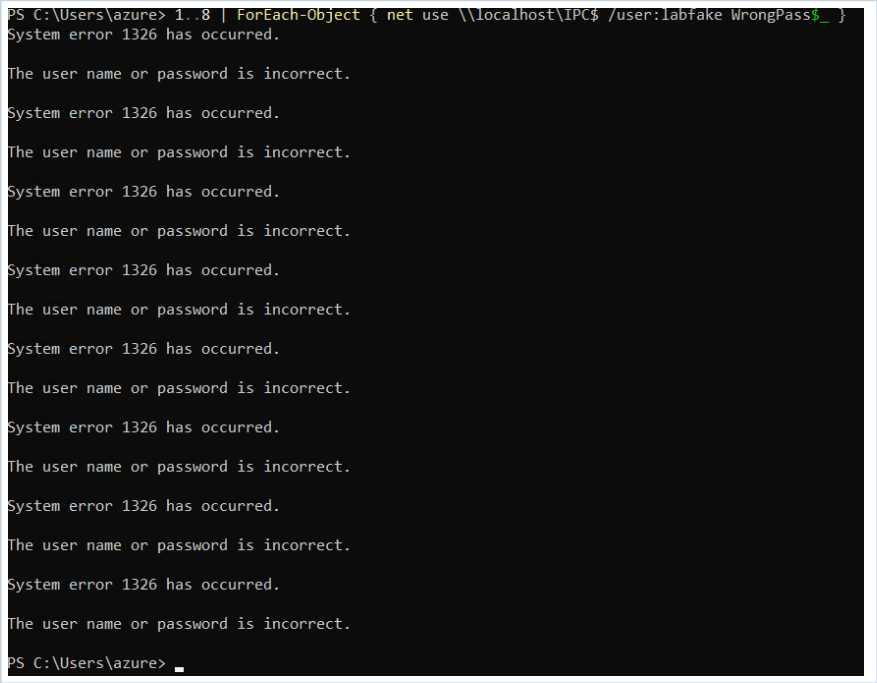


Figure 13. *Each attempt fails with error 1326, "The user name or password is incorrect."*

What happened. Windows wrote every failure into its log as event 4625. A few minutes later, the rule noticed the pattern and Sentinel created an incident. First I checked that the raw events really were in the workspace:

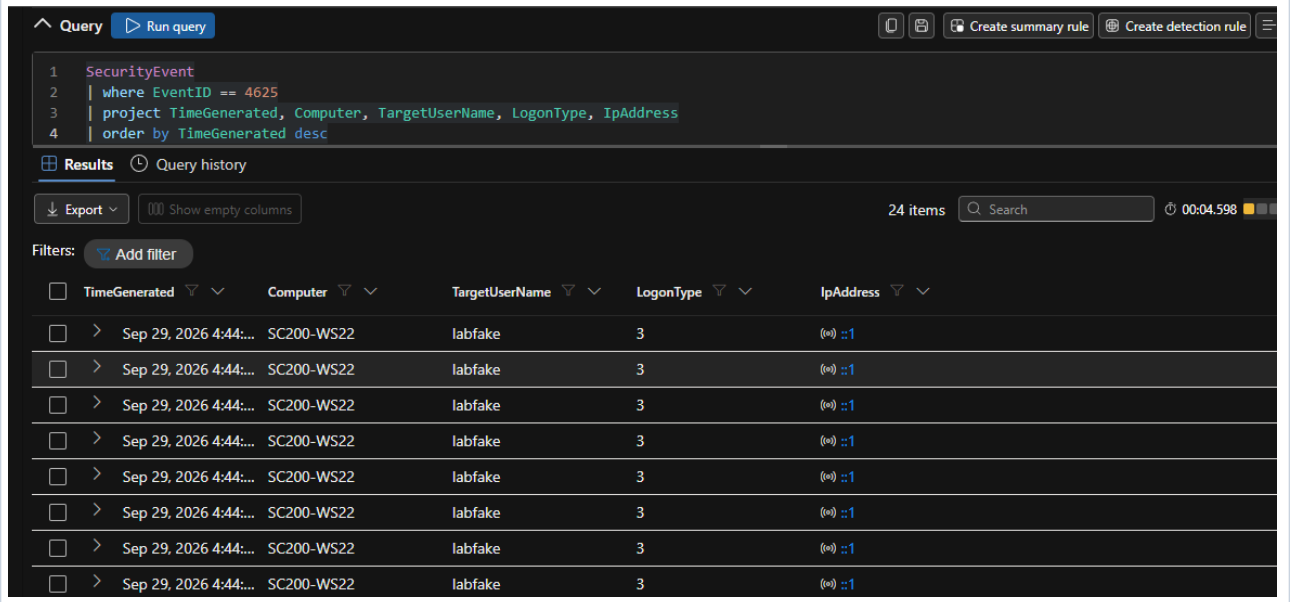


Figure 14. *The failed sign-ins in the logs: same account (labfake), same computer, logon type 3 (a network sign-in). At this point the logs held 24 events from my test runs.*

Reading the alert. The alert page shows the evidence the rule found: the account, the computer, and how many failures it counted. It also shows that the server belongs to my SC200-Lab-Servers device group.

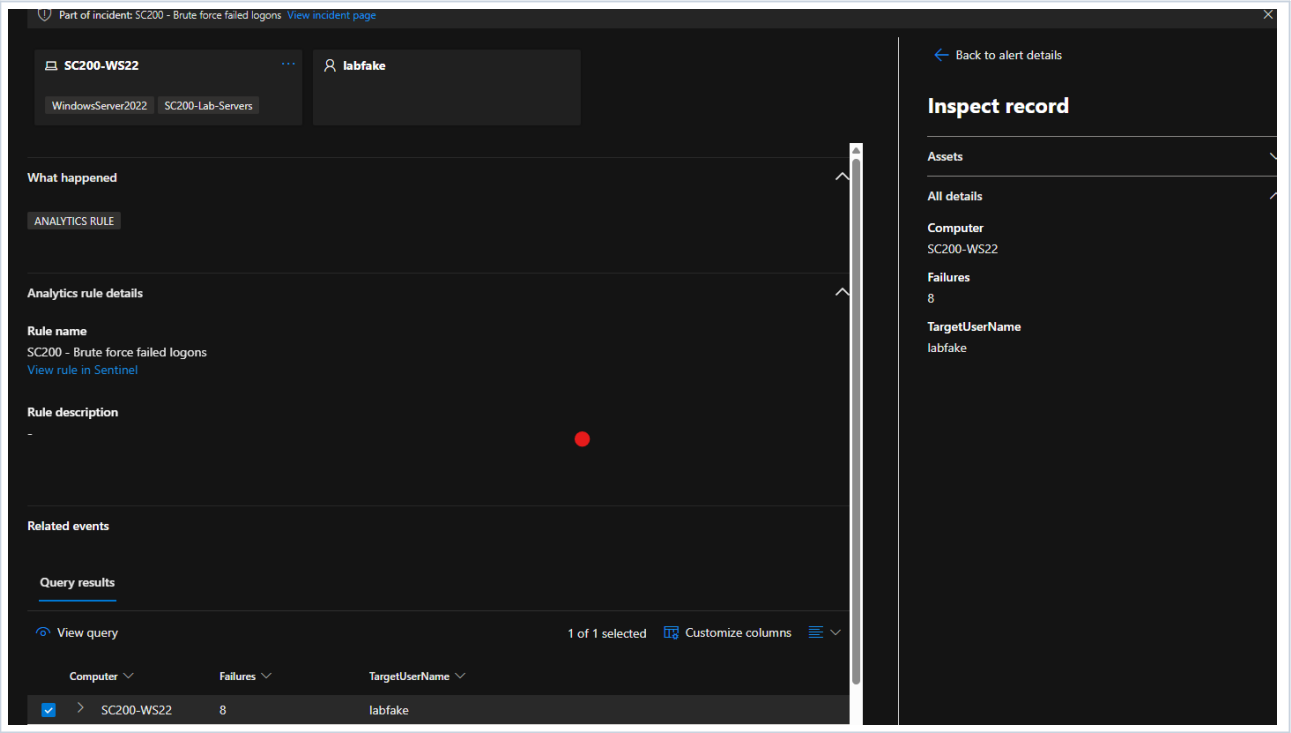


Figure 15. The alert: the fake account labfake, the computer SC200-WS22 with its device-group label, and the failure count (8 in this run).

How I investigated it:

- Confirmed the alert named the right computer and the right account, and that the number of failures matched what I did.
- Noticed that the sign-ins came from **::1**. That is the computer talking to itself (localhost). It makes sense, because I attacked from inside. A real attack from outside would show an outside address.
- Opened the **Assets** tab to see which computer and which account were linked to the incident.

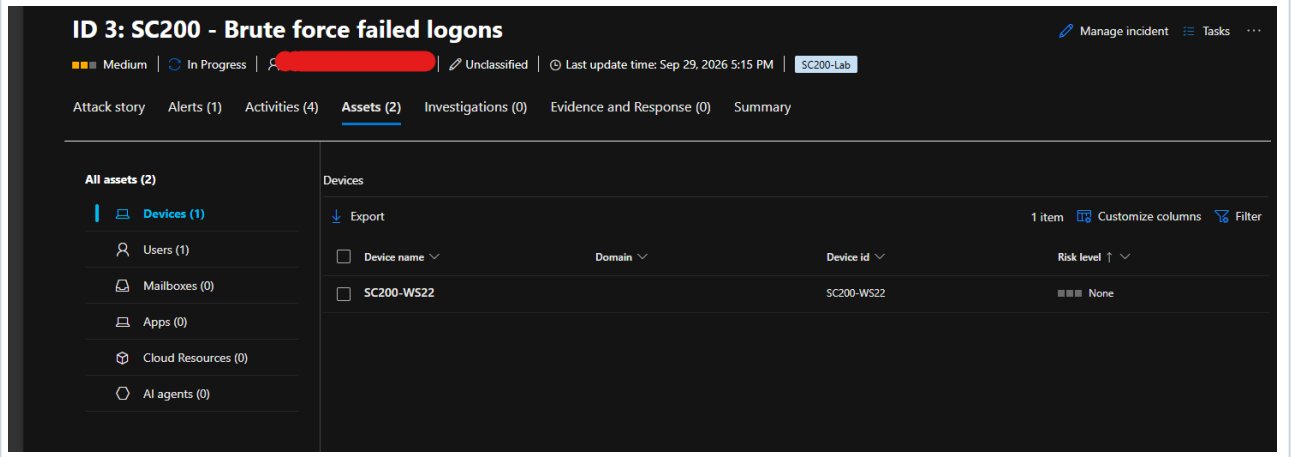


Figure 16. The incident's Assets tab: the host and the account tied to this incident.

Closing it properly. An incident is not finished until it has a classification and a note. I set the status to **Resolved**, chose **Informational, expected activity: Security testing**, and wrote a comment saying exactly what this was.

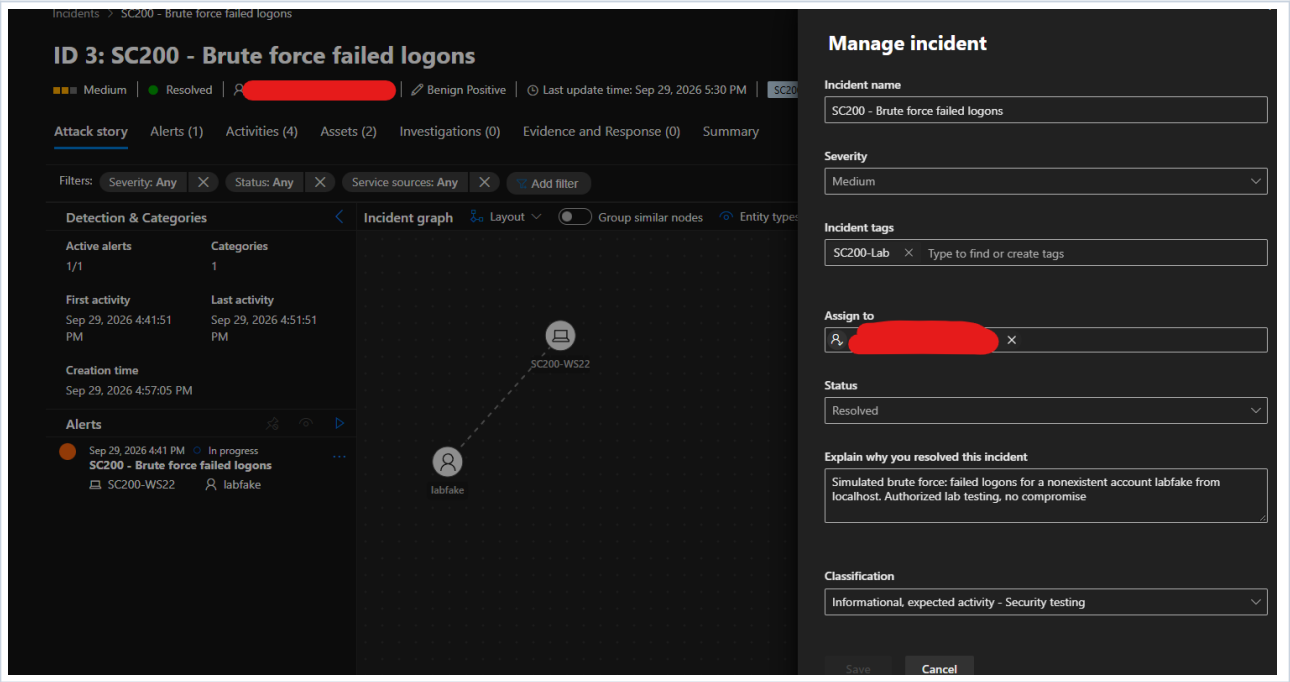


Figure 17. The incident closed with its classification and my explanation.

> **The three classification groups, in plain words**
>
> **True positive:** a real attack. **Informational, expected activity:** real activity that is allowed, like my own testing. **False positive:** the alarm was wrong. Picking the right group matters, because it teaches the team which alarms to trust.

ATTACK 2

Creating a hidden admin account

What I did. I created a new local user called **labtemp** and added it to the **Administrators** group. Attackers do this to keep a secret way back into a machine after they are discovered. (I used a throwaway password, shown as a placeholder below.)

```
net user labtemp <test-password> /add
net localgroup administrators labtemp /add
```

What Sentinel showed. A **High** severity incident from my second rule, tagged with the Persistence tactic.

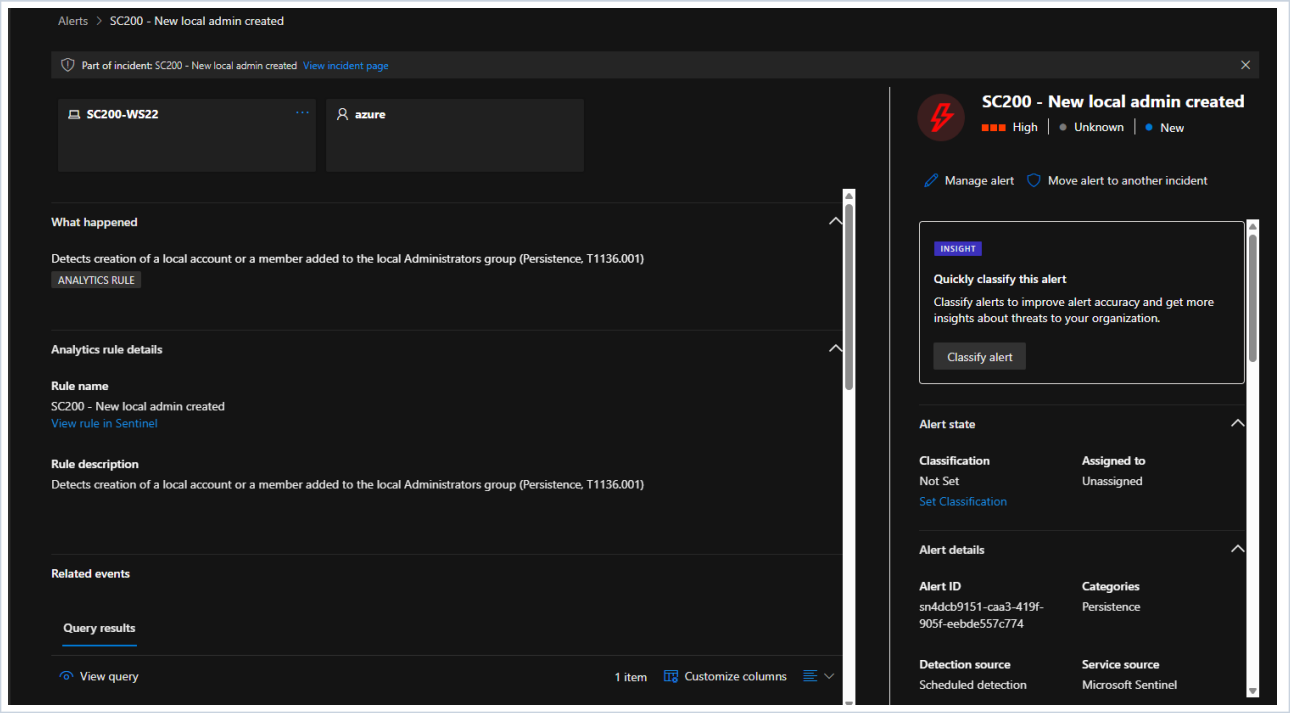


Figure 18. The alert "SC200 - New local admin created": High severity, Persistence, with the host and the account that made the change.

The evidence row says: event **4732**, a member was added to a security-enabled local group, the group is **Administrators**, and the change was made by the account **azure**.

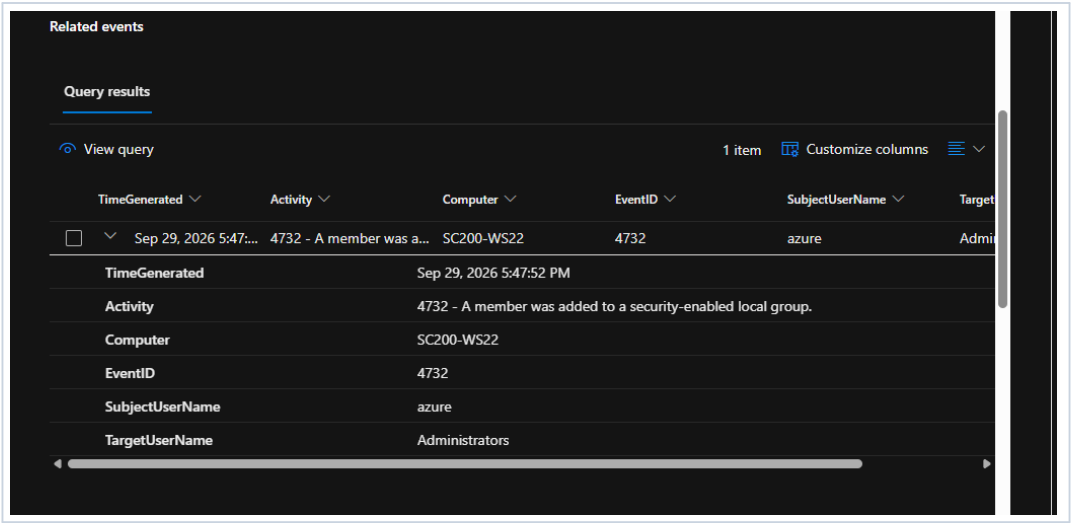


Figure 19. The evidence behind the alert: event 4732, group Administrators, performed by azure.

The puzzle. The alert told me *who made the change* and *which group*, but not *which account was added*. In a real investigation, that is the question that matters most. So I went to the raw logs.

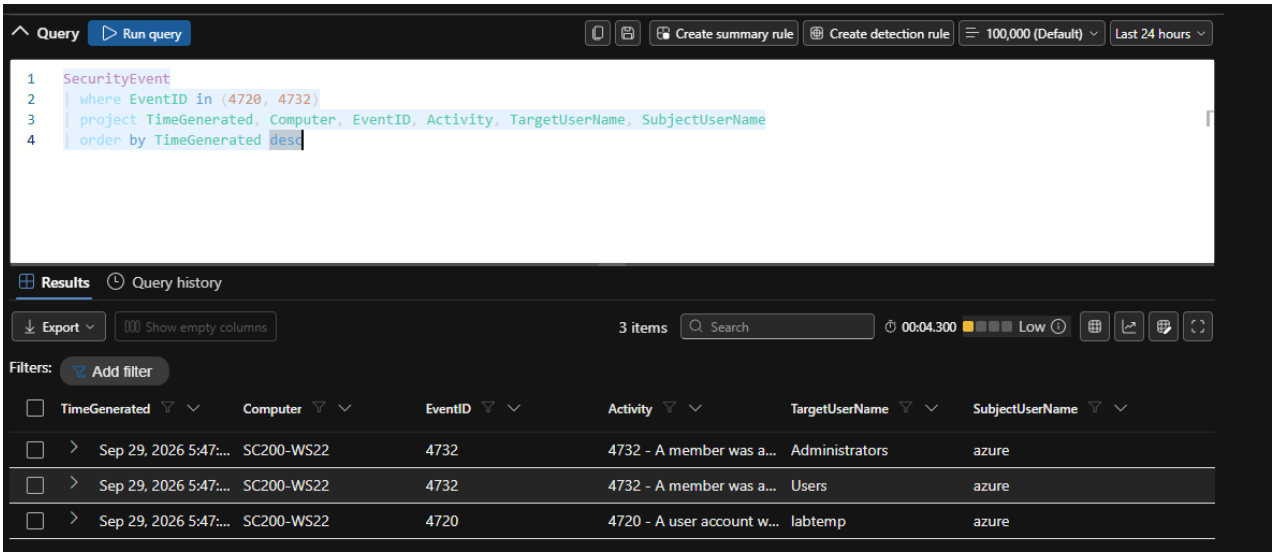


Figure 20. *The raw events in Advanced hunting: one 4720 (account labtemp created) and two 4732 (added to groups).*

There were two 4732 events: one for **Administrators** and one for **Users**. The second is harmless. Windows adds every new account to the Users group automatically, so I treated it as normal.

The fix: connect the two events by SID. Every Windows account has a unique ID number called a **SID**, a bit like a roll number. The "account created" event (4720) and the "added to group" event (4732) both carry that number, so I joined them on it:

```
let created = SecurityEvent
    | where EventID == 4720
    | project NewAccount = TargetUserName, Sid = TargetSid;
SecurityEvent
| where EventID == 4732 and TargetUserName == "Administrators"
| project AddedTime = TimeGenerated, Computer, ActedBy = SubjectUserName, Sid = MemberSid
| join kind=inner created on Sid
| project AddedTime, Computer, ActedBy, NewAccount, Sid
```

The result: one row. The account **labtemp** was added to **Administrators** by **azure**, on **SC200-WS22**, at 5:47 PM. Case closed.

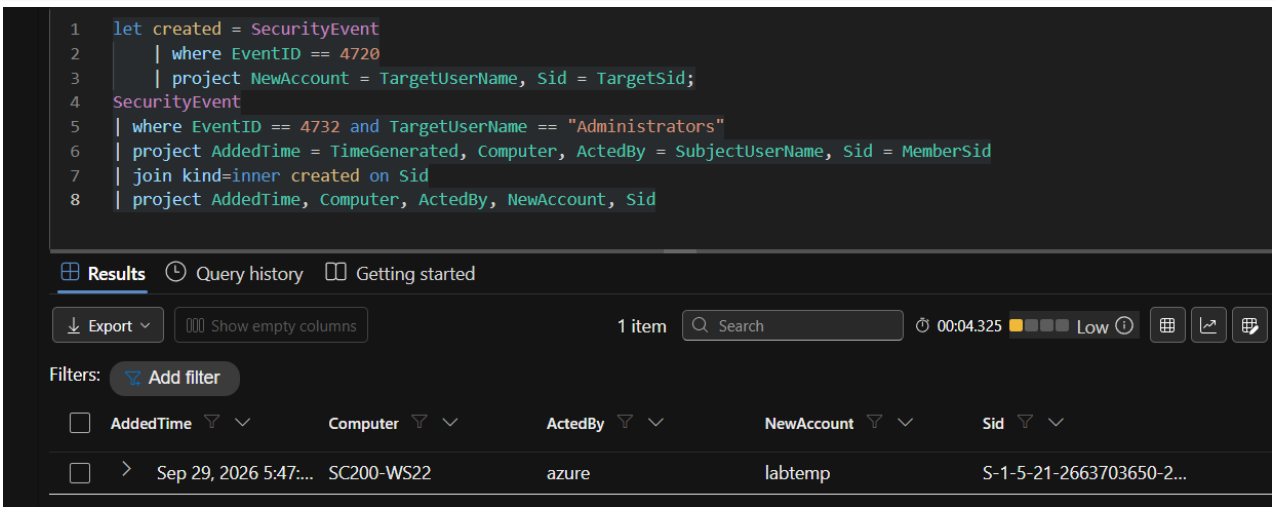


Figure 21. *The join returns a single row that answers the whole question.*

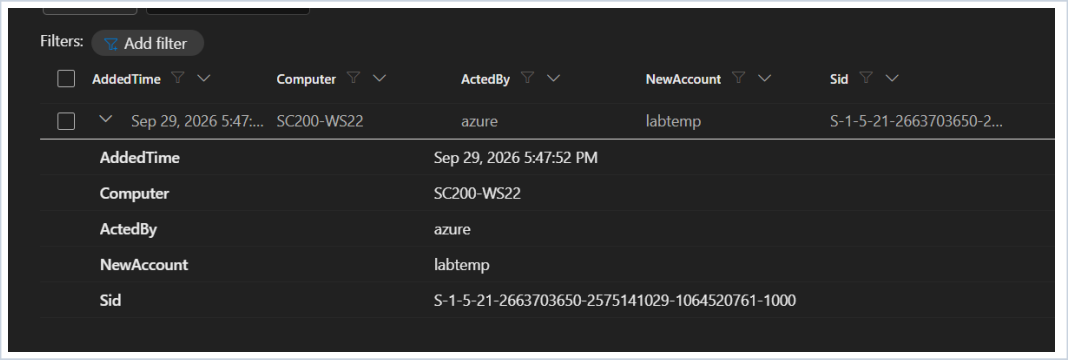


Figure 22. *The same row expanded, showing the time, computer, who did it, the new account, and the full SID.*

Clean-up. I closed the incident the same way as the first one, then deleted the labtemp account and checked the user list to make sure it was gone.

GOOD HABITS

Keeping cost and risk low

This lab ran on a free trial, so I treated it like a real budget. Here is what I did:

- **Locked RDP to one address,** so strangers could not touch the server.
- **Collected only the Common events,** not everything, to keep log storage small.
- **Turned on a single paid Defender plan** (Servers) and left the rest off.
- **Stopped the VM properly after every session.** "Stopped (deallocated)" means Azure stops charging for the computer. Just shutting Windows down is not enough.

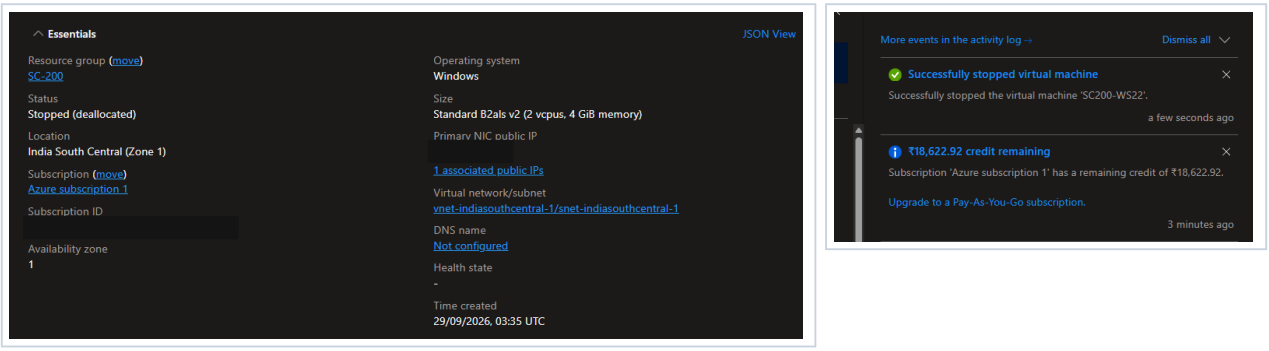


Figure 23. *The VM shown as Stopped (deallocated), with the portal confirming the stop. Subscription ID and public address are hidden.*

REFLECTION

What I learned

- **Tell Sentinel what each field is.** My first rule put the username inside the Host entity. Moving it into its own Account entity made the incident graph and the Assets tab show the computer and the person correctly.
- **An alert is a starting point, not the answer.** The admin alert did not name the new account. I had to hunt through the raw events to find it.
- **Good evidence fields save time.** If rule 2 also output the member's SID, the new account would be visible in the alert itself.
- **Test traffic looks different from real traffic.** Attacking from inside the server showed ::1 as the source. A real attack from outside would show a real address.
- **Repeating a test repeats the alerts.** Running the same attack several times created several alerts. In a real team, alert grouping is how you keep that manageable.
- **Two tool families, two clock names.** Sentinel tables use TimeGenerated. Defender hunting tables use Timestamp. Checking which one you are in avoids confusing query errors.

Skills this project shows

SC-200 area	What I did in this lab
Manage a security operations environment	Connected a server to Sentinel with the Azure Monitor Agent and a data collection rule. Wrote scheduled analytics rules with MITRE and entity mapping. Enabled Defender for Servers, connected the device, and set up a device group with an approval-based remediation level.
Respond to security incidents	Triaged two incidents: read the alert, reviewed the assets and evidence, assigned and commented, then classified and closed each one.
Perform threat hunting	Used KQL in Sentinel and Advanced hunting to summarise failed sign-ins, inspect raw events, and join two events by SID to identify the account behind a privilege change.

Words explained

Word	What it means
Alert	A single warning raised by a detection rule.
Incident	One or more related alerts bundled together for an analyst to investigate.
Microsoft Sentinel	Microsoft's cloud security tool that collects logs and raises alerts (a SIEM).
Defender XDR	Microsoft's security portal that brings together alerts from servers, identities, email, and more.
Defender for Endpoint	Software that watches a computer for suspicious behaviour.
Log Analytics workspace	The cloud storage area where the logs are kept and searched.
Data Collection Rule	A list that says which logs to collect and where to send them.
Azure Monitor Agent	The helper program on the server that sends logs out.
KQL	Kusto Query Language: the search language used in Sentinel and Defender.
MITRE ATT&CK	A public catalogue of attacker tactics and techniques.
Entity mapping	Telling Sentinel which fields in an alert are a computer, an account, or an address.
RDP	Remote Desktop Protocol, used to sign in to a server remotely (port 3389).
Network security group	A list of rules that allow or block network traffic to a machine.
SID	A unique ID number that Windows gives every account.
Event 4625 / 4720 / 4732	Windows security events: failed sign-in / account created / member added to a local group.